

Lecture 12

Thunks, Promises, Streams

Thanks

```
let my_if (c : bool) (t : 'a) (e : 'a) : 'a =  
  match c with  
  | true  -> t  
  | false -> e
```

my_if e1 e2 e3 = if e1 e2 e3 ?

Delayed Evaluation

- Arguments to function calls are evaluated **eagerly** (before call)
- The branches of an if are evaluated **lazily** (not eager)

Thunks

- Can delay computation by wrapping in an “unnecessary” function
 - Only purpose is to delay evaluation
- Zero argument function used to delay evaluation called thunks

Thunks

- Can delay computation by wrapping in an “unnecessary” function
 - Only purpose is to delay evaluation
- Zero argument function used to delay evaluation called thunks

```
let my_if (c : bool) (t : unit -> 'a) (e : unit -> 'a) : 'a =  
  match c with  
  | true -> t ()  
  | false -> e ()
```

Thunks and Semantics

- If the thunk is *pure* and there is no *mutation*, then when the body executes only affects performance + termination
- With mutation, when things evaluate may affect the result

Thunks and Performance

- Delaying computation can help, hurt, or not change performance
 - If you never end up needing the result: **help**
 - If you need the result more than once: **hurt**
 - If you need the result exactly once: **no change**
- It can be hard to predict how many times you will need a result!

Promises

The Best of Both Worlds

- Often a better approach:
 - Don't compute the result **until it is needed**
 - Once you need it, **save the answer and reuse it** if need it again
- This is a form of **lazy evaluation** (“call by need”)
 - Default behavior in languages like Haskell
 - Can be simulated with *promises*
 - Actually built into Racket
 - But we will make our own out of thunks and mutable pairs to understand the mechanics

Promises

- ADT w/ operations **delay** and **force**
- One implementation: a mutable pair of a bool and “something”
 - The bool indicates if answer has already been computed
 - If true, the second component is the saved answer
 - If false, the second component is a thunk that gives the answer

Promises and Performance

- Like thunks, faster if result not used
- Like thunks, no change if result used exactly once
- *Unlike thunks, faster if result used more than once!*

Group Work

```
let l = ref [ 1; 2; 3 ]
let f1 () = List.fold_left ( + ) 0 !l
let f2 = delay (fun () -> List.fold_left ( + ) 0 !l)
```

```
let res1 = f1 ()
let res2 = force f2
```

```
let _ = l := [ 4; 5; 6 ]
```

```
let res3 = f1 ()
let res4 = force f2
```

What do res1, res2, res3, and res4 evaluate to?

```
(* delay : thunk -> promise *)
let delay (th : 'a thunk) : 'a promise =
  ref (Unevaluated th)

(* force : promise -> answer *)
let force (p : 'a promise) : 'a =
  match !p with
  | Unevaluated th ->
    let x = th () in
    p := Evaluated x;
    x
  | Evaluated a -> a
```

Streams

Streams

- Often want abstraction of infinite sequence of values (network, UI, ...)
 - Cannot compute or store them all, only those you need!
 - Use a thunk to represent “the rest of the stream”
 - Essentially infinite lists, implemented as an **idiom** using features we have already seen
- The *producer* and *consumer* collaborate, sort of like a 2 player game
 - Producer knows how to create all values
 - Consumer decides how many values to ask for

Streams

- We will define a stream as a thunk that returns an option:
 - None if the stream is terminated
 - Some containing a pair:
 - The “head of the stream” at that point
 - A stream of the rest of the values

```
type 'a stream = Stream of (unit -> 'a * 'a stream) option
```


Implementing Streams

- A stream is a thunk that returns a pair option:
 - None if the stream is terminated
 - Some pr
 - **fst pr** is the “head of the stream” at that point
 - **snd pr** is a stream of the rest of the values
- Producer must be prepared to generate any number of elements!
 - But how?? **Recursion of course!**
 - The key “gotcha” is to thunk in just the right place

Example Streams

```
let rec ones = Stream (fun () -> Some (1, ones))
```

```
let nats =
```

```
  let rec go start = Some (start, Stream (fun () -> go (start + 1))) in  
  Stream (fun () -> go 0)
```

Group Work

Write a function `add_streams` that takes two `int streams` and returns an `int stream` where each element of the result stream is the sum of the corresponding elements of the input streams.

If either stream terminates, the result stream should also terminate.

